



تقرير مراجعة تطبيق المحلل القانوني المصري

مراجعة شاملة للتصميم والكود والأمان والأداء

تاريخ المراجعة: 20 مايو 2026

نسخة التطبيق: 2024/2025

نوع الملف: HTML ملف واحد (Single Page Application)

مراجعة فنية شاملة

فهرس المحتويات

1. نظرة عامة على التطبيق

1.1 الفكرة والهدف

1.2 التقنيات المستخدمة

2. تصميم واجهة المستخدم (UI/UX)

2.1 نقاط القوة في التصميم

2.2 نقاط الضعف في التصميم

3. هيكل الكود وجودته

3.1 نقاط القوة في الكود

3.2 نقاط الضعف في الكود

4. قاعدة البيانات

4.1 التقييم الحالي

4.2 التوصيات

5. الأمان والحماية

5.1 الثغرات الأمنية الحرجة

5.2 التوصيات الأمنية

6. الأداء والتحسين

7. إمكانية الوصول

8. التكامل مع واجهات البرمجة (API)

9. جدول النتائج والتوصيات

10. الخلاصة والتقييم العام

1. نظرة عامة على التطبيق

1.1 الفكرة والهدف

تطبيق "المحلل القانوني المصري" هو أداة ويب تفاعلية مصممة لمساعدة المحامين والباحثين القانونيين في مصر واستخراج الدفوع الشكلية والموضوعية والثغرات الإجرائية بالإضافة إلى مبادئ أحكام محكمة النقض التطبيق أيضا ميزة صياغة مسودات مذكرات قانونية جاهزة للاقتباس، وهي ميزة عملية توفر وقتا كبيرا في إعداد، يستهدف التطبيق خمسة فروع قانونية رئيسية هي: قانون العقوبات، قانون الإجراءات الجنائية، القانون المدني، الأحوال الشخصية. ويوفر لوحة إدارة لاستيراد النصوص القانونية الكاملة من مصادر خارجية عبر أداة crawl بخدمات الذكاء الاصطناعي للتحليل المتقدم.

1.2 التقنيات المستخدمة

يعتمد التطبيق على بنية تقنية بسيطة تتكون من ملف HTML واحد يحتوي على جميع الأكواد المضمنة (script مكتبة Tailwind CSS عبر CDN للترتيب، وخطوط Google Fonts (Amiri و Tajawal) للنصوص العربية. يتصل التطبيق وهي: `/api/public/article` لجلب نص المادة، و `/api/public/analyze` للتحليل، و `/api/public/ingest-law` لاستيراد القوانين، بالإضافة إلى نقطة `/api/public/law-stats` لعرض إحصائيات القوانين.

ملاحظة: التطبيق في حالته الحالية يُعد نموذجا أوليا (Prototype) يعتمد على بيانات ثابتة مدمجة في الكود مع واجهات برمجة خلفية، لكنه ليس تطبيقا إنتاجيا جاهزا للاستخدام الفعلي دون تحسينات جوهرية.

2. تصميم واجهة المستخدم (UI/UX)

2.1 نقاط القوة في التصميم

الهوية البصرية المتسقة: يتميز التطبيق بتصميم بصري متناسق يعكس الطابع القانوني الرسمي من خلال (ذهبي) تمنح شعورا بالثقة والاحترافية. الألوان المستخدمة متجانسة ومتناسقة مع طبيعة التطبيق القانوني.

العميق للرسمية والجدية، بينما يضيف اللون الذهبي لمسة من الرفعة والمكانة. هذا الاختيار اللوني ناجح جدا في للمستخدم منذ اللحظة الأولى.

دعم اللغة العربية: التطبيق يدعم الاتجاه من اليمين لليسار (RTL) بشكل صحيح عبر خاصية `dir="rtl"` استخدام خطوط عربية مناسبة (Tajawal للنصوص العامة و Amiri للعناوين). هذا يضمن عرضا سليما للمحتوى؛ محاذاة النص أو ترتيب العناصر. كما أن اختيار خط Amiri للعناوين يمنحها طابعا كلاسيكيا يتناسب مع الطبيعة

التصميم المتجاوب: يستخدم التطبيق شبكة CSS Tailwind المتجاوبة (grid system) مع فئات `md:grid-cols-12` و `lg:grid-cols-2`، مما يضمن عرضا مناسباً على مختلف أحجام الشاشات من الشاشات الكبيرة. هذا يعكس وعياً جيداً بضرورة التوافق مع أجهزة متعددة.

التصنيف البصري الواضح: يستخدم التطبيق شارات ملونة (Badges) مميزة لكل نوع من أنواع الدفوع؛ البنفسجي للدفوع الموضوعية، الأحمر للثغرات، والذهبي لأحكام النقض. هذا الترميز اللوني يسهل على المسد الفئات المختلفة ويحسن تجربة القراءة بشكل ملحوظ.

الأمثلة السريعة: توفير أزرار أمثلة سريعة (عقوبات 17، إجراءات 304، مدني 163) يسهل على المستخدم الجيد دون الحاجة لمعرفة أرقام المواد القانونية مسبقاً، وهو ما يحسن تجربة المستخدم الأولية بشكل كبير.

2.2 نقاط الضعف في التصميم

غياب حالات التحميل المتقدمة: رغم وجود مؤشرات تحميل بسيطة (رمز الساعة الرملية)، إلا أن التطبيق يمتحركة (Spinners أو Skeleton Loading) تمنح المستخدم إحساساً أفضل بأن النظام يعمل. مؤشر "جارٍ التحمى" وحده قد لا يكون كافياً خاصة عند بطء الاتصال، حيث قد يظن المستخدم أن التطبيق توقف عن الاستجا

غياب التنقل بين النتائج: لا توجد طريقة للتنقل بين نتائج بحث سابقة أو حفظ سجل البحث. عند إجراء السابقة نهائياً دون إمكانية الرجوع إليها. هذا يمثل مشكلة عملية للمحامي الذي يحتاج للمقارنة بين عدة مواد ق

تصدير محدود: ميزة التصدير تنتج ملف نصي بسيط (TXT) فقط، بينما البيئة القانونية تحتاج صيغاً أكثر احترافاً تنسيق مناسب يتضمن شعار المكتب والبيانات الأساسية للمذكرة. الملف النصي البسيط لا يليق بطبيعة الد مظهراً رسمياً ومهنية عالية في الوثائق.

لوحة الإدارة المكشوفة: لوحة استيراد القوانين مرئية لجميع المستخدمين رغم استخدام عنصر `<details>` الاحترافية، يجب أن تكون لوحة الإدارة في صفحة منفصلة محمية بمصادقة، وليست ضمن الواجهة الرئيسية ا حتى وإن كان مطوياً يسبب ارتباكاً للمستخدم العادي ويمثل مخاطرة أمنية.

غياب التعامل مع الأخطاء البصري: عند فشل الاتصال بالخادم، تظهر رسائل خطأ نصية بسيطة داخل الة مناسب يلفت الانتباه. يجب أن تظهر الأخطاء في إشعارات منبثقة (Toast Notifications) أو صناديق تنبيه واضحة

3. هيكل الكود وجودته

3.1 نقاط القوة في الكود

البنية الوظيفية الواضحة: الكود منظم بشكل منطقي مع فصل واضح بين الأقسام: الأنماط (CSS) في رأس والمنطق البرمجي (JavaScript) في نهايتها. التعليقات العربية المستخدمة لتقسيم الأقسام (مثل "محرك البحث القانونية") تسهل قراءة الكود وفهم بنيته. هذا التنظيم يجعل الصيانة والتعديل أسهل رغم كون الكود في ملف

الدوال المساعدة المفيدة: دوال مثل `fill()` و `fillNaqd()` و `notFoundData()` توفر إعادة استد التكرار. دالة `notFoundData()` بالذات توفر بيانات نموذجية ذكية عندما لا تكون المادة موجودة في قاعدة البيانات من التعطل ويمنح المستخدم نتائج ولو تقريبية.

التراجع المتدرج (Graceful Degradation): التطبيق مصمم ليعمل حتى بدون اتصال بالخادم، حيث يلجأ للبيانات فشل الاتصال بـ API. هذه ميزة مهمة تضمن استمرار عمل التطبيق في مختلف الظروف. كل نقطة اتصال بـ API تتعامل مع الأخطاء بسلسلة.

3.2 نقاط الضعف في الكود

ملف واحد ضخم (Monolithic Architecture): جميع الأكواد (HTML + CSS + JS + قاعدة البيانات) في ملف واحد هذا النمط غير مناسب للتطبيقات الحقيقية لأنه يصعب صيانته واختباره وتطويره من قبل فريق عمل. الفصل (فصل بسيط) أصبح معياراً أساسياً في تطوير الويب الحديث، وتجاهله يحد من قابلية التوسع بشكل كبير.

ثغرة XSS خطيرة: الدالة `fill()` تستخدم `innerHTML` لحقن المحتوى مباشرة:

```
items.map(i => `- ${i}</li>`).join('');

```

هذا النمط يعرض التطبيق لهجوم حقن الشيفرات المتقاطعة (Cross-Site Scripting). إذا تم العبث بالبيانات المحلية أو من استجابة API، يمكن تنفيذ شيفرات خبيثة في متصفح المستخدم. الحل الآمن هو استخدام العناصر عبر `document.createElement()` مع تعيين النص بشكل آمن.

عدم التحقق من المدخلات: حقل رقم المادة لا يخضع لأي تحقق من صحة المدخلات (Input Validation). النص بما في ذلك شيفرات خبيثة. يجب تطبيع المدخلات (Sanitization) والتحقق من أنها تتطابق مع نمط أرقام مثل استخدام تعبير نمطي يتحقق من أن الإدخال يحتوي فقط على أرقام أو أرقام مع حروف محددة.

استخدام CDN في الإنتاج: الاعتماد على `cdn.tailwindcss.com` في رأس الصفحة يمثل مشكلة على د CDN التطويري في بيئات الإنتاج لأنه بطيء ويحتاج تحميل في كل زيارة. ثانيا، إذا تعطل خادم CDN أو حُجب ف التطبيق تنسيقه بالكامل. يجب تجميع (Build) ملفات CSS مسبقا وضمها محليا.

غياب اختبارات الوحدة: لا توجد أي اختبارات آلية (Unit Tests) للمنطق البرمجي. مع تطور التطبيق وإضافة د الاختبارات مخاطرة حقيقية قد تؤدي لانحدار الكود (Regression) دون اكتشافه. هذا أمر بالغ الأهمية خاصة ف الأخطاء فيه مكلفة.

4. قاعدة البيانات

4.1 التقييم الحالي

قاعدة البيانات الحالية مدمجة في الكود ككائن JavaScript ثابت (Object) يحتوي على خمسة إدخالات فقط هي: 234 عقوبات، المادة 304 إجراءات جنائية، المادة 206 إجراءات جنائية، والمادة 163 مدني. هذا يعني أن التغطيا ضئيلة جدا من إجمالي المواد القانونية المصرية، والتي تُعد بآلاف المواد عبر القوانين الخمسة المدعومة.

من الناحية الإيجابية، بنية كل إدخال في قاعدة البيانات مصممة بشكل ممتاز ومتكامل، حيث تتضمن: نصr الشكلية، والدفع الموضوعية، والثغرات، وأحكام النقص مع أرقام الطعون، ومسودة المذكرة القانونية. هذا يمكن البناء عليه لتوسيع قاعدة البيانات بشكل منهجي. كما أن الدالة `notFoundData()` توفر بيانات ال الموجودة، مما يمنع التطبيق من التعطل.

ومع ذلك، فإن تخزين قاعدة البيانات ككود JavaScript ثابت يعني أن أي تحديث يتطلب تعديل الكود المصr بالكامل. هذا النمط غير عملي للتطبيقات الديناميكية التي تحتاج تحديث بياناتها باستمرار خاصة أن التشريع دوري مع إصدار تعديلات جديدة.

4.2 التوصيات

- نقل قاعدة البيانات إلى قاعدة بيانات حقيقية (PostgreSQL أو MongoDB) مع واجهة API مستقلة للوصول ل منطق البيانات ومنطق العرض.
- بناء نظام استيراد آلي يمكنه جلب وتحليل النصوص القانونية من المصادر الرسمية (مثل منشورات.أورج) بد هيكلها.
- إضافة آلية تخزين مؤقت (Caching) محلية باستخدام IndexedDB أو localStorage لتحسين سرعة الاستجابة الشبكة.

- توسيع قاعدة البيانات لتشمل المواد الأكثر استخداما أولا (مبدأ باريتو: 20% من المواد تُستخدم في 80% من تدريجيا.

5. الأمان والحماية

5.1 الثغرات الأمنية الحرجة

تحذير: التطبيق يحتوي على ثغرات أمنية خطيرة يجب معالجتها قبل أي استخدام فعلي في بيئة إنتاجية.

الثغرة	الخطورة	الوصف	الحل المقترح
حقن الشيفرات المتقاطعة (XSS)	حرج	استخدام innerHTML لحقن بيانات ديناميكية بدون تطبيع يسمح بتنفيذ شفرات خبيثة في متصفح المستخدم عبر حقن وسوم script أو معالجات أحداث onerror	استخدام ent DOM بشكل آ المدخلات قبل
رمز الاستيراد بدون حماية	حرج	رمز الاستيراد (INGEST_TOKEN) يُرسل كنص صريح في الطلب بدون أي آلية مصادقة حقيقية، ويمكن اعتراضه بسهولة	استخدام نظام OAuth مع إدارة آمنة للـ
غياب حماية CSRF	عالي	طلبات POST لواجهة الاستيراد لا تحتوي على أي رمز حماية ضد تزوير الطلبات بين المواقع	إضافة رموز n الطلبات المع منها على الخا
عدم التحقق من المدخلات	عالي	حقن رقم المادة يقبل أي إدخال بدون تحقق، مما يمكن استغلاله لحقن شفرات في طلبات API	تطبيق تحقق والخدم مع ت الأحرف المس
غياب تحديد معدل الطلبات	متوسط	لا يوجد حد لعدد الطلبات الممكن إرسالها لواجهات API، مما يعرض الخادم لهجمات الحرمان من الخدمة	تطبيق niting النهاية مع عت طلب
لوحة الإدارة مكشوفة	متوسط	وظائف الإدارة (استيراد القوانين) متاحة في الواجهة الرئيسية بدلا من صفحة إدارية محمية منفصلة	فصل لوحة الإ بمصادقة وت

5.2 التوصيات الأمنية

- **إصلاح XSS فورا:** استبدال جميع استخدامات `innerHTML` بدائل آمنة. هذا الإصلاح يجب أن يكون الأولو خطرا مباشرا على مستخدمي التطبيق.
- **تطبيق مبدأ الدفاع المتعدد الطبقات:** التحقق من المدخلات على مستوى العميل والخادم معا، وتطبيق معالجة أو عرضها.
- **تشفير جميع الاتصالات:** ضمان استخدام HTTPS لجميع الاتصالات مع الخادم، خاصة عند إرسال رموز المالحسة.
- **تطبيق سياسة أمن المحتوى (CSP):** إضافة رؤوس Content-Security-Policy لتقييد مصادر السكريبتات و المسموح بتحميلها.
- **إدارة أمنة للجلسات:** تطبيق نظام مصادقة وجلسات آمن مع انتهاء صلاحية الجلسات تلقائيا بعد فترة من

6. الأداء والتحسين

يعاني التطبيق من عدة مشاكل في الأداء قد تؤثر على تجربة المستخدم خاصة عند زيادة حجم البيانات وعدد المد تحميل **Tailwind عبر CDN:** مكتبة Tailwind CSS تُحمل بالكامل من CDN في كل زيارة، مما يعني تحميل مئات ال المستخدمة. في بيئة الإنتاج، يجب تجميع (Build) فقط الأنماط المستخدمة فعليا باستخدام أداة PostCSS مع يمكن أن يقلل حجم CSS من أكثر من 3 ميجابايت إلى أقل من 50 كيلوبايت.

غياب التخزين المؤقت (Caching): التطبيق لا يستخدم أي آلية للتخزين المؤقت محليا. كل بحث عن مادة للخادم حتى لو سبق البحث عن نفس المادة في نفس الجلسة. استخدام IndexedDB أو حتى localStorage لت يمكن أن يقلل حجم الطلبات الشبكية بشكل كبير ويحسن سرعة الاستجابة، خاصة للمواد التي يكثر البحث عنها.

عدم تحميل الخطوط بشكل مسبق: خطوط Google Fonts تُحمل بدون استخدام `font-display: swap` قد يسبب وميض النص (FOUT) أو تأخرا في عرض المحتوى. يجب إضافة `rel="preconnect"` لـ `fonts.gstatic.com` لتسريع تحميل الخطوط وتحسين تجربة العرض الأولي.

قاعدة بيانات محلية كبيرة: مع توسع قاعدة البيانات المحلية (كائن DB)، سيزداد حجم ملف HTML الأساس تحميل الصفحة الأولى. يجب نقل البيانات بالكامل إلى واجهة API واستخدام التحميل الكسول (Lazy Loading) الحاجة.

عدم ضغط الصور والأصول: التطبيق لا يستخدم أي تقنيات ضغط أو تحسين للأصول. عند إضافة أي صور يجب استخدام صيغ حديثة مثل WebP مع أحجام متعددة (Responsive Images) وضغط بدون فقدان.

7. إمكانية الوصول

إمكانية الوصول (Accessibility) في التطبيق ضعيفة وتحتاج تحسينات جوهرية لتتوافق مع معايير WCAG 2.1 الم

المشكلة	الخطورة	التأثير	الحل
غياب سمات ARIA	عالي	قارئات الشاشة لا تستطيع فهم بنية الصفحة والأزرار التفاعلية والمناطق الديناميكية	إضافة label لجميع العناصر المحتوى الديناميكي
غياب تنقل لوحة المفاتيح	عالي	المستخدمون الذين يعتمدون على لوحة المفاتيح يواجهون صعوبة في التنقل بين الأقسام واستخدام التطبيق	إضافة styles ترتيب التبوب اختصارات لوحة
تباين ألوان غير كافٍ	متوسط	بعض النصوص الرمادية الفاتحة على خلفية داكنة قد تكون صعبة القراءة لضعاف البصر	زيادة نسبة التباين للنصوص الع
غياب النصوص البديلة	متوسط	الأيقونات SVG والرموز التعبيرية ليس لها نصوص بديلة لقارئات الشاشة	إضافة label SVG alt للنصوص
عدم إدارة التركيز	متوسط	عند ظهور النتائج، لا ينتقل التركيز تلقائياً إليها مما يربك مستخدمي قارئات الشاشة	نقل التركيز بر بعد تحميلها aria-live

8. التكامل مع واجهات البرمجة (API)

التطبيق مصمم للتكامل مع ثلاث واجهات برمجة رئيسية، لكن التنفيذ الحالي يواجه عدة تحديات:

واجهة جلب نص المادة (/api/public/article)

تعمل بشكل مقبول مع معالجة أخطاء بسيطة عبر try/catch. المشكلة الرئيسية هي غياب التحقق من صحة الـ (Response Validation). إذا أُرجع الخادم بيانات ببنية غير متوقعة، قد يتعطل التطبيق بصمت. يجب إضافة طبقة قبل استخدام البيانات، مع التأكد من وجود الحقول المطلوبة ونوعها الصحيح.

واجهة التحليل بالذكاء الاصطناعي (/api/public/analyze)

تمثل الواجهة الأكثر أهمية وقيمة في التطبيق، لكن التنفيذ الحالي يرسل نص المادة الكامل إلى API بدون أي بالنسبة للمواد القانونية الطويلة، هذا يعني طلبات شبكية كبيرة وأحمال معالجة عالية على الخادم. ينبغي إرس فقط (رقم المادة وفرع القانون والنص) مع تجنب البيانات الزائدة. كما يجب إضافة آلية إعادة المحاولة (Exponential Backoff) عند فشل الطلب.

واجهة استيراد القوانين (/api/public/ingest - law)

هذه الواجهة هي الأكثر خطورة أمنياً. رمز المصادقة يُرسل كرأس HTTP بسيط (x-ingest-token) بدون أ من الهوية. كما أن URL المصدر (sourceUrl) يُرسل بدون تحقق، مما يفتح باب هجمات SSRF (تزوير الطلب يمكن للمهاجم إجبار الخادم على طلب موارد داخلية. يجب التحقق من أن URL المصدر ينتمي للنطاقات المسموح ورفض أي طلبات لـ URLs خارجية غير معتمدة.

واجهة إحصاءات القوانين (/api/public/law-stats)

واجهة بسيطة لعرض عدد المواد المحفوظة لكل قانون. لا توجد مشاكل أمنية كبيرة فيها، لكن يُنصح بإضاف لتقليل الحمل على الخادم عند الطلبات المتكررة، خاصة أن هذه البيانات لا تتغير بشكل متكرر.

9. جدول النتائج والتوصيات

م	المجال	الأولوية	المشكلة	التوصية
1	الأمان	حرج	ثغرة XSS عبر innerHTML	استبدال erHTML بـ createElement
2	الأمان	حرج	غياب مصادقة حقيقية لمواجهة الاستيراد	تطبيق نظام مصدقات للجلسات
3	الأمان	عالي	عدم التحقق من المدخلات	تطبيق تحقق صارم مع تطبيع المدخلات
4	الأمان	عالي	غياب حماية CSRF	إضافة رموز CSRF
5	الكود	عالي	بنية ملف واحد ضخمة	فصل الملفات إلى استخدام إطار عمل
6	البيانات	عالي	قاعدة بيانات محدودة ومدمجة في الكود	نقل إلى قاعدة بيانات مستقلة
7	الأداء	عالي	تحميل Tailwind كاملاً من CDN	تجميع CSS محلياً المطلوبة
8	التصميم	متوسط	تصدير نصي بسيط فقط	إضافة تصدير DF احترافي
9	التصميم	متوسط	لوحة الإدارة مكشوفة في الواجهة الرئيسية	فصلها في صفحة مستقلة
10	التصميم	متوسط	غياب سجل البحث والتنقل بين النتائج	إضافة سجل بحث متعدد
11	الأداء	متوسط	غياب التخزين المؤقت	استخدام indexedDB مؤقتاً

م	المجال	الأولوية	المشكلة	التوصية
12	إمكانية الوصول	عالي	غياب سمات ARIA ودعم قارئ الشاشة	إضافة aria-label للعناصر التفاعلية
13	الأمان	متوسط	غياب تحديد معدل الطلبات	تطبيق Limiting ٣ النهاية
14	الكود	منخفض	غياب اختبارات آلية	إضافة اختبارات و Vitest

10. الخلاصة والتقييم العام

5.5

من 10 - تقييم عام

التقييم التفصيلي حسب المجال

الفكرة والقيمة المضافة

تصميم واجهة المستخدم (UI/UX)

هيكل الكود وجودته

قاعدة البيانات

الأمان والحماية

الأداء والتحسين

يتميز تطبيق "المحلل القانوني المصري" بفكرة مبتكرة وقيمة مضافة حقيقية للمجتمع القانوني المصري، حيث المحامين والباحثين في تحليل المواد القانونية واستخراج الدفوع والثغرات بشكل آلي. التصميم البصري احتر التطبيق، ودعم اللغة العربية متميز.

مع ذلك، التطبيق في حالته الحالية يُعد نموذجاً أولياً يحتاج إلى عمل جوهري قبل أن يكون صالحاً للاستخدام الثغرات الأمنية (خاصة XSS وغياب المصادقة) تمثل المخاطرة الأكبر ويجب معالجتها كأولوية قصوى. كما أن البيانات المحدودة تحدان بشكل كبير من قابلية التوسع والصيانة على المدى الطويل.

الخلاصة: الفكرة ممتازة والتصميم البصري جيد، لكن التنفيذ الفني يحتاج تحسينات جذرية في الأمان وهيكلي ليصبح التطبيق جاهزاً للاستخدام الحقيقي. ننصح بمعالجة الثغرات الأمنية الحرجة أولاً، ثم إعادة هيكلة البيانات وتحسين الأداء.